

Worksheet 8: Numerical Optimization

What is an Objective Function?

An objective function is a function that we want to minimize or maximize. Examples in statistics:

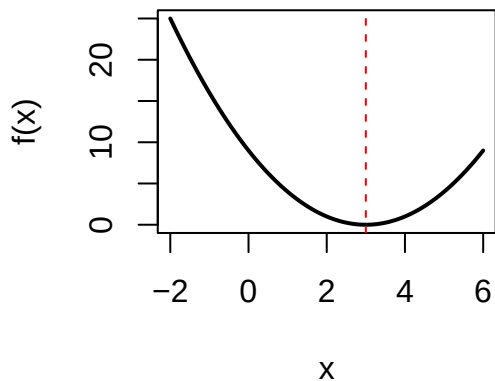
- likelihood function in maximum likelihood estimation
- least squares in least squares regression
- KL distance minimization in tSNE
- and many/many more

Example: minimize

$$f(x) = (x - 3)^2$$

Simple Example

```
f <- function(x){  
  (x - 3)^2  
}  
curve(f(x), -2, 6, lwd=2)  
abline(v=3, col="red", lty=2)
```



Why Do We Need Numerical Methods?

In most statistical model fitting, the maximization or minimization cannot be done in closed form. Let us consider the following example to understand. Let $X_1, X_2, \dots, X_n \stackrel{iid}{\sim} \text{Cauchy}(\theta, 1)$. Recall the pdf of Cauchy is

$$f(x_i) = \frac{1}{\pi} \frac{1}{1 + (x_i - \theta)^2}.$$

If we want to estimate θ using maximum likelihood estimator, then our objective function is

$$L(\theta|\mathbf{x}) = f(x_1, x_2, \dots, x_n|\theta) = \prod_{i=1}^n f(x_i|\theta).$$

Then notice that

$$\arg \max L(\theta|x) = \arg \max \underbrace{\log L(\theta|x)}_{l(\theta|x)} = \arg \max \sum_{i=1}^n \log f(x_i|\theta).$$

Now, if we try to maximize this, we will naturally want to take a derivative and set to 0. When I do this, I get the following:

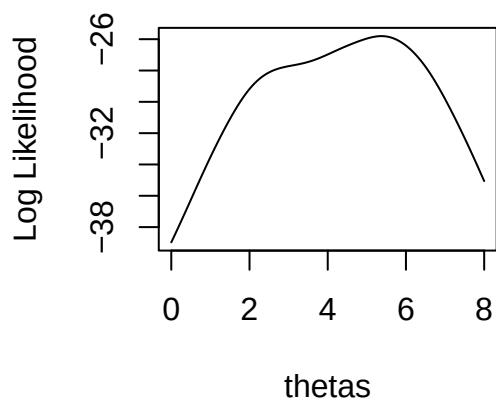
$$\frac{dl}{d\theta} = \sum_{i=1}^n \frac{2(x_i - \theta)}{1 + (x_i - \theta)^2} = 0.$$

As hard as you try, you cannot solve the above to obtain a solution. Further, $l(\theta)$ is NOT a concave function! Which might create other problems. Let's see the objective function by simulating from Cauchy!

```
set.seed(3)
n <- 10
theta <- 5
x <- rcauchy(n = n, location = theta)

# let's plot the likelihood function
thetas <- seq(0, 8, length = 1e3)
log.like <- sapply(thetas, function(t)
  sum(dcauchy(x, location = t, log = TRUE)) )

plot(thetas, log.like, type = 'l', ylab = "Log Likelihood")
```



Nelder-Mead method

Consider an objective function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ that we want to minimize. The Nelder-Mead method finds a minima by “trapping” it inside a simplex. Recall that a simplex in d -dimensions has $d + 1$ vertices. The algorithm runs in the following way:

Define parameters:

- $\alpha = 1$ (reflection)
- $\gamma = 2$ (expansion)
- $\rho = 1/2$ (contraction)
- $\sigma = 1/2$ (shrink)

Start with vertices $\{\theta_1, \theta_2, \dots, \theta_{d+1}\}$ and assume they are ordered by the objective function values:

$$f(\theta_1) \leq \dots \leq f(\theta_{d+1}).$$

1. Order the vertices as above.
2. Calculate the centroid:

$$c = \frac{1}{d} \sum_{i=1}^d \theta_i$$

The centroid is the center of the d good vertices.

3. **Reflection:** Reflect the worst point across the centroid: $\theta_r = c + \alpha(c - \theta_{d+1})$.
 - a. If $f(\theta_1) \leq f(\theta_r) < f(\theta_d)$ (the reflected point is better), then set $\theta_{d+1} = \theta_r$.
 - b. If $f(\theta_r) < f(\theta_1)$ (then, we might be able to do even better continuing in this direction, so)
 - i. Calculate $\theta_e = c + \gamma(\theta_r - c)$ (go beyond θ_r)
 - ii. If $f(\theta_e) < f(\theta_r)$, then $\theta_{d+1} = \theta_e$
 - iii. Else $\theta_{d+1} = \theta_r$.
 - c. If $f(\theta_r) \geq f(\theta_d)$ (then going beyond may not help, so contract)
 - a. Calculate $\theta_c = c + \rho(\theta_r - c)$
 - i. If $f(\theta_c) < f(\theta_{d+1})$, then set $\theta_{d+1} = \theta_c$.
 - ii. Else, shrink the simplex: so for all $i = 2, \dots, d+1$, set

$$\theta_i = \theta_1 + \sigma(\theta_i - \theta_1)$$

4. Repeat until the $\max_{i,j} \|\theta_i - \theta_j\| \leq \text{tolerance}$ that is provided.

When the algorithm stops, it returns θ_1 of the last simplex upon termination.

Other methods

There are many other optimization techniques, particularly those that employ gradients and Hessians. You will learn about them in Statistical Computing.

Questions

1. Write your own version of the Nelder-Mead function assuming the function is one-dimensional. Your function should look like:

```
# fn: a function that can take inputs
# x0: starting value for the algorithm
# tol: tolerance for simplex size
# maxit: maximum number of iterations
```

```
nelder_mead <- function(fn, x0, alpha = 1, gamma = 2, rho = 0.5,
                        sigma = 0.5, tol = 1e-8, maxit = 1000)
{
  # return best objective value
}
```

2. Homework: try to write your own function for Nelder Mead for any dimension!
3. Test your function for $f(x) = (x - 3)^2$.
4. Find the MLE of the Cauchy example using your function.
5. The `optim` function in R implements this method (as well as other optimization techniques). Use the `optim` function to solve question 4 again.
6. Consider the Rosenbrock function $f(x, y) = (1 - x)^2 + 100(y - x^2)^2$. Plot the contours of this function using the command below:

```
# rosenbrock function
## Note for Nelder-Mead, this version cannot work,
## since the function must have a single argument
rosenbrock <- function(x, y){
  (1 - x)^2 + 100*(y - x^2)^2
}
x <- seq(-2, 2, length = 200)
y <- seq(-1, 3, length = 200)

# evaluate rosenbrock on these grid of values
z <- outer(x, y, rosenbrock)
# contours of the function
contour(x, y, z,
        nlevels = 40,
        xlab = "x",
        ylab = "y",
        main = "Rosenbrock Function")

## Nelder-Mead compatible function
rosenbrock <- function(z)
{
  x <- z[1]
  y <- z[2]
  (1 - x)^2 + 100*(y - x^2)^2
}
```

7. Find the minima of the Rosenbrock function.